



Especialización en Inteligencia Artificial

Algoritmos evolutivos I

Desafío práctico

Panadería 24 horas

Alumno

Ing. Juan Cruz Ojeda

Docente

Esp. Ing. Miguel Augusto Azar

Fecha: 17 de Octubre de 2025

Tabla de Contenidos

Tabla de Contenidos	1
Resumen	2
1. Introducción	2
2. Planteo del problema	2
3. Formulación	3
3.1 Representación	3
3.2 Función objetivo	3
3.3 Restricciones	4
4. Método Evolutivo	4
4.1 Representación e inicialización	4
4.2 Evaluación	4
4.3 Selección, cruza y mutación	4
4.4 Elitismo y registro	4
4.5 Parámetros y parada	5
5. Experimentos	5
5.1 Configuración	5
5.2 Convergencia	5
5.3 Incidencias y solución aplicada	6
5.4 Calidad de solución	6
6. Resultados y análisis	7
7. Métodos del código fuente	7
8. Referencias	8

Resumen

Se modela un problema de asignación de personal en una panadería que opera 24 h, con demanda por hora no uniforme. Se propone una formulación como minimización de un objetivo escalar que combina costo laboral y penalizaciones por violar restricciones (disponibilidad, cobertura, límites diarios y de consecutividad), con bonificación por continuidad de turnos. Se resuelve mediante un Algoritmo Genético implementado con DEAP. Se reporta la convergencia del fitness en un gráfico y uno de los tres mejores resultados.

Enlace al desafío en [Google Colab](#)

Enlace al repositorio en [Github](#)

1. Introducción

La planificación de personal en un servicio continuo de 24 horas se formula como un problema de asignación con demanda horaria variable y múltiples restricciones operativas. En el caso estudiado, una panadería debe cubrir cada hora del día al menor costo laboral posible, respetando la disponibilidad de cada empleado, los límites máximos de horas por día y de horas consecutivas, y evitando patrones indeseados de cobertura. También se considera un adicional para el trabajo nocturno.

Este trabajo modela el problema mediante una función objetivo que combina costo y penalizaciones por violación de restricciones, junto con una bonificación por continuidad de turnos. Se emplea una codificación por slots de cobertura en la que cada unidad de demanda se representa como un gen, lo que permite incorporar de forma directa la condición de cobertura mínima por hora. La evaluación empírica se realiza mediante curvas de convergencia del fitness obtenidas a partir de la evolución generacional.

2. Planteo del problema

- Horizonte: 24 horas (0 a 23).
- Demanda: cantidad de personas requeridas por hora, con picos en la mañana y en la tarde/noche.

- Empleados: lista de personas, cada una con costo por hora y una franja de disponibilidad horaria.
- Decisión del algoritmo: para cada unidad de demanda (slot) en cada hora, asignar qué empleado la cubre.
- Criterios y restricciones:
 - cubrir la demanda de cada hora.
 - no asignar fuera de la disponibilidad declarada.
 - respetar máximos de horas por día y de horas consecutivas por empleado.
 - evitar bloques aislados de 1 hora y el patrón 1-0-1.
 - considerar un adicional para el trabajo nocturno.

3. Formulación

3.1 Representación

El individuo se representa como una lista de enteros. Cada posición corresponde a un slot de cobertura, es decir, a una unidad de demanda en una hora determinada. El valor almacenado en esa posición indica qué empleado toma ese slot. Un arreglo auxiliar relaciona cada slot con su hora para poder computar costos y restricciones.

3.2 Función objetivo

El valor a minimizar se compone de tres partes:

1. Costo laboral: suma del costo por hora de cada empleado asignado, incluyendo un adicional para horas nocturnas.
2. Penalizaciones: aumentan el valor cuando se violan reglas prácticas del negocio:
 - a. asignar a alguien fuera de su franja disponible.
 - b. exceder el máximo de horas diarias.
 - c. superar el máximo de horas consecutivas.
 - d. dejar slots sin cubrir.
 - e. generar bloques aislados de una hora o el patrón 1-0-1 (dos bloques separados por una hora libre).
 - f. asignar dos veces al mismo empleado en una misma hora.
3. Bonificación por continuidad: premia las asignaciones que encadenan horas consecutivas para la misma persona, favoreciendo turnos más compactos y realistas.

Los pesos de cada penalización y bonificación se ajustaron de forma empírica hasta lograr coberturas completas sin costos excesivos.

3.3 Restricciones

En lugar de imponer restricciones duras, se modelan como penalizaciones en la función objetivo. Se verifica cobertura mínima por hora, disponibilidad individual, límites por día y por consecutividad, y se desincentivan patrones indeseados. Este esquema permitió mantener la búsqueda flexible y guiar al algoritmo hacia soluciones lo más factibles posibles.

4. Método Evolutivo

Se implementó un algoritmo genético utilizando la biblioteca DEAP en Python.

4.1 Representación e inicialización

La población inicial se genera de forma aleatoria uniforme sobre el conjunto de empleados disponibles para cada slot, lo que permite explorar diversas combinaciones de cobertura desde el inicio.

4.2 Evaluación

El fitness es escalar y corresponde a la función objetivo descrita en la [Sección 3.2](#), que combina costo laboral, penalizaciones por violaciones de reglas operativas y una bonificación por continuidad de turnos. Esta formulación evita restricciones duras y guía la búsqueda hacia soluciones factibles.

4.3 Selección, cruza y mutación

La selección se realiza por torneo de tamaño 5. La cruza es de dos puntos. La mutación es uniforme con probabilidad de cambio $\text{indpb}=0.1$ por gen, lo que introduce diversidad suficiente sin desestabilizar la población.

4.4 Elitismo y registro

Se utiliza un *Hall Of Fame* (salón de la fama) de tres individuos para preservar las mejores soluciones encontradas. Por generación se registran el mejor, el promedio y el peor fitness, lo que permite trazar la curva de convergencia y monitorear la estabilidad de la búsqueda.

4.5 Parámetros y parada

En los experimentos se empleó una población de 500 individuos, 100 generaciones, probabilidad de cruce 0,8 y de mutación 0,2. El criterio de parada es un número fijo de generaciones.

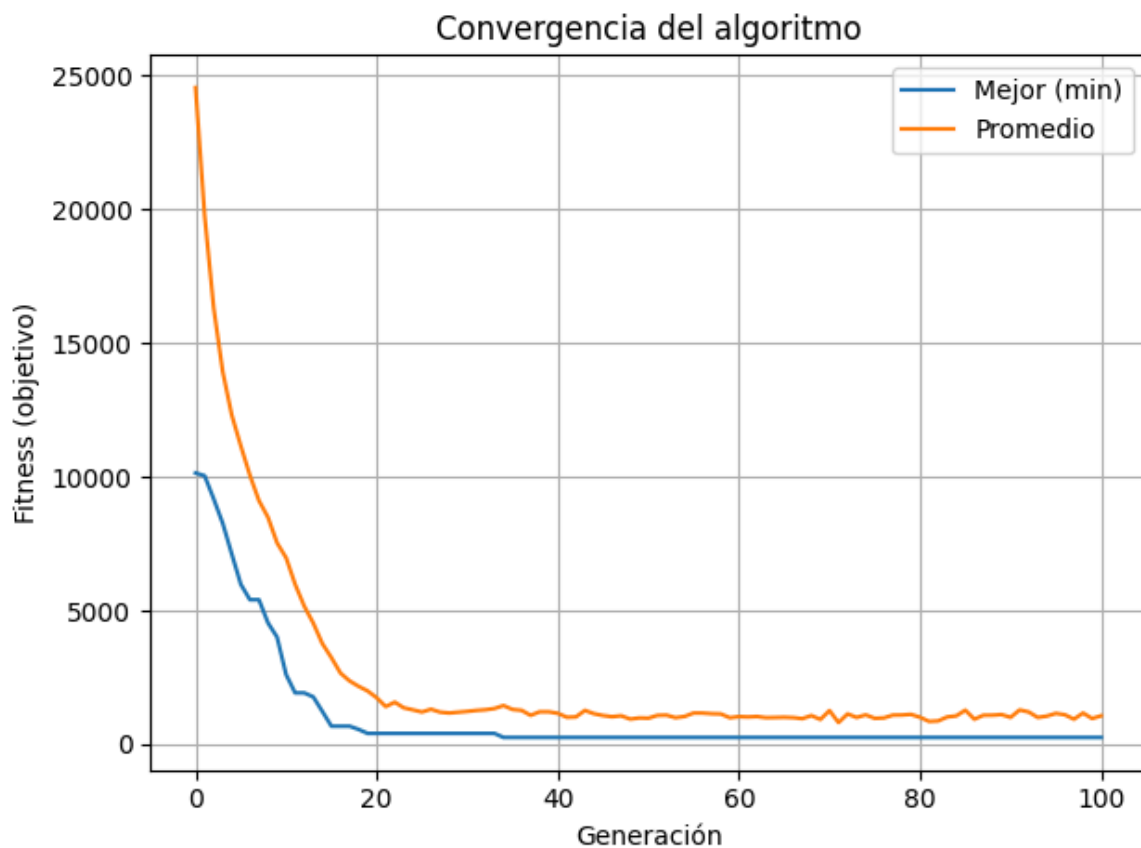
5. Experimentos

5.1 Configuración

Se ejecutó el algoritmo con población de tamaño 500 durante 100 generaciones y se registraron, por generación, el mejor, el promedio y el peor valor de fitness.

5.2 Convergencia

La curva de convergencia muestra la evolución del mejor y del promedio del fitness por generación. Se observa la fase inicial de mejora rápida seguida por una estabilización.



5.3 Incidencias y solución aplicada

Durante el desarrollo se detectó un problema recurrente: la aparición del patrón 1-0-1, esto es, asignaciones con una hora libre entre dos horas trabajadas para la misma persona. Este patrón es indeseable operativamente porque fragmenta turnos y dificulta la organización. Para mitigarlo, se incorporó una penalización específica sobre dicho patrón, con un peso suficiente para que el algoritmo prefiriera turnos más compactos. Tras ajustar ese peso, el patrón dejó de aparecer en las mejores soluciones.

5.4 Calidad de solución

Se reportan los valores de la función objetivo de una de las tres mejores soluciones y su distribución de horas por empleado. En general, se logra cubrir la demanda en picos horarios sin exceder los límites diarios ni de consecutividad, y con buena continuidad de turnos.

```
=====
SOLUCIÓN 1 (Objetivo: 259.444)
=====
00:00-01:00 | Demanda=1 | Asignados: Carla
01:00-02:00 | Demanda=1 | Asignados: Carla
02:00-03:00 | Demanda=1 | Asignados: Carla
03:00-04:00 | Demanda=1 | Asignados: Carla
04:00-05:00 | Demanda=1 | Asignados: Carla
05:00-06:00 | Demanda=1 | Asignados: Ana
06:00-07:00 | Demanda=3 | Asignados: Eva, Ana, Diego
07:00-08:00 | Demanda=3 | Asignados: Diego, Ana, Eva
08:00-09:00 | Demanda=3 | Asignados: Diego, Eva, Ana
09:00-10:00 | Demanda=3 | Asignados: Ana, Eva, Diego
10:00-11:00 | Demanda=3 | Asignados: Ana, Diego, Fedé
11:00-12:00 | Demanda=1 | Asignados: Fedé
12:00-13:00 | Demanda=1 | Asignados: Diego
13:00-14:00 | Demanda=1 | Asignados: Diego
14:00-15:00 | Demanda=1 | Asignados: Diego
15:00-16:00 | Demanda=1 | Asignados: Fedé
16:00-17:00 | Demanda=1 | Asignados: Fedé
17:00-18:00 | Demanda=3 | Asignados: Bruno, Eva, Diego
18:00-19:00 | Demanda=3 | Asignados: Bruno, Eva, Diego
19:00-20:00 | Demanda=3 | Asignados: Eva, Fedé, Bruno
20:00-21:00 | Demanda=3 | Asignados: Eva, Bruno, Fedé
21:00-22:00 | Demanda=1 | Asignados: Carla
22:00-23:00 | Demanda=2 | Asignados: Carla, Fedé
23:00-00:00 | Demanda=2 | Asignados: Fedé, Carla
Horas por empleado:
  Ana: 6 h
  Bruno: 4 h
  Carla: 8 h
  Diego: 10 h
  Eva: 8 h
  Fedé: 8 h
```

6. Resultados y análisis

Los resultados muestran una mejora sostenida del mejor y del promedio del fitness durante las primeras generaciones hasta alcanzar una meseta estable, lo que indica convergencia del algoritmo con los parámetros elegidos. En las soluciones destacadas se cumple la cobertura horaria sin asignaciones fuera de disponibilidad y respetando los límites diarios y de consecutividad. La incorporación de penalizaciones específicas para bloques aislados de una hora y, en particular, para el patrón 1-0-1, redujo la fragmentación de turnos y favoreció secuencias más compactas sin aumentar de forma significativa el costo total. El término de equidad y la bonificación por continuidad contribuyeron a distribuir las horas entre empleados de manera más balanceada, evitando concentraciones excesivas.

7. Métodos del código fuente

La función `evaluar(individuo)` calcula el fitness escalar sumando el costo horario de cada asignación y aplicando penalizaciones por violaciones operativas (asignar fuera de disponibilidad, exceder horas diarias o consecutivas, dejar demanda sin cubrir, duplicar a la misma persona en la misma hora) y una bonificación por continuidad.

La función `crear_individuo()` define la representación: un cromosoma como lista de enteros donde cada gen es un "slot" de demanda asignado a un empleado.

El operador de cruce se implementa en `crossover_individuos(ind1, ind2)` mediante `tools.cxTwoPoint`. Este cruce de dos puntos recombina bloques contiguos del horario, favoreciendo la herencia de patrones útiles como tramos consecutivos de trabajo, que suelen ser deseables a nivel operativo.

La mutación se implementa en `mutar_individuo(individuo, indpb=0.1)`, que recorre los genes y, con probabilidad `indpb`, reasigna el "slot" a otro empleado. Esta perturbación controlada introduce diversidad y ayuda a escapar de óptimos locales sin desestabilizar la población.

La selección se registra en el toolbox con `tools.selTournament`, que elige individuos por torneos aleatorios.

Finalmente, en `main()` se construyen la población inicial, el `HallOfFame` y las estadísticas (`tools.Statistics`) que recopilan por generación los valores min,

avg y max del fitness. La llamada a `algorithms.eaSimple(..., stats=stats, halloffame=hof, verbose=True)` ejecuta el ciclo evolutivo y devuelve el `logbook`, que luego se utiliza para generar el gráfico de convergencia.

8. Referencias

- DEAP: *Distributed Evolutionary Algorithms in Python*.
- Notas de cátedra Algoritmos Evolutivos (UBA/CEIA).